

# MAD 2 Project Report

## Student Details

**Name:** Syed Ali Mujtaba

**Roll Number:** 21f1004482

**Email:** [21f1004482@ds.study.iitm.ac.in](mailto:21f1004482@ds.study.iitm.ac.in)

**About Me:** I am a 24-year-old software engineer at HSBC Technology India in Pune, where I focus on backend development and testing using SpringBoot and RestAssured frameworks. Alongside my professional work, I am pursuing the IITM BS degree. I am a dedicated and adaptable learner.

---

## Project Details

**Project Title: Vehicle Parking App - V2**

### Description

This project implements a **Vehicle Parking Management System** that enables seamless interaction between **Admins and Users** through a unified web interface. It allows parking spot booking, reservation management, and history tracking while ensuring role-based access control and automated notifications via background jobs.

### Approach

A modular full-stack implementation:

- **Backend:** Flask API with SQLAlchemy ORM and JWT auth for secure, stateless sessions. Business rules cover spot exclusivity and billed-hours with a minimum charge.
  - **Frontend:** Vue 3 SPA (Vite) with axios, router guards, Bootstrap utilities, and Chart.js for analytics.
  - **Background jobs:** Celery workers (Redis broker/backend) handle CSV exports, daily reminders, and monthly PDF reports, keeping HTTP requests responsive.
  - **Caching:** Redis caches public lots and analytics to reduce DB load and improve perceived latency.
- 

## Technologies Used

| Technology / Library      | Purpose                                                                            |
|---------------------------|------------------------------------------------------------------------------------|
| <b>Flask</b>              | Backend web framework for building REST APIs and serving the application.          |
| <b>Flask-JWT-Extended</b> | Provides secure token-based authentication and role-based access control.          |
| <b>SQLAlchemy</b>         | ORM used for database table mapping and operations.                                |
| <b>SQLite</b>             | Lightweight, file-based database for persistent storage.                           |
| <b>Celery</b>             | Handles asynchronous and scheduled background tasks (exports, reminders, reports). |
| <b>Redis</b>              | Used as an in-memory store for caching and as a Celery message broker/backend.     |
| <b>Vue 3 (Vite)</b>       | Modern frontend framework for building interactive and responsive SPA.             |
| <b>Vue Router</b>         | Client-side routing with navigation guards for protected routes.                   |
| <b>Axios</b>              | HTTP client for API communication.                                                 |
| <b>Chart.js</b>           | Data visualization library for analytics charts.                                   |
| <b>Bootstrap 5</b>        | For consistent, responsive, and modern UI design.                                  |
|                           |                                                                                    |

## AI/LLM Declaration

Throughout the development of this Vehicle Parking Management System project, AI tools (such as **ChatGPT (GPT-5)**) were used selectively to assist in certain development phases — accounting for approximately **40–50%** of the overall project work.

The detailed breakdown of AI involvement is as follows:

### **Vue.js Frontend Development:** ~25%

- Used for guidance on Vue 3 composition API patterns, component structure, and reactive data binding.
- AI assistance helped with router guard implementation and Chart.js integration patterns.

### **Backend Development:** ~10%

- Used to improve code readability, variable naming conventions, and identifying potential edge cases.
- AI suggestions were manually reviewed and adapted to project requirements.

### **Testing and Debugging:** ~3–5%

- Used to identify and resolve bugs, form validation issues, and API integration problems.
- AI assistance helped troubleshoot Celery task scheduling and Redis caching logic.

#### Background Jobs (Celery): ~10%

- Consulted for correct Celery task configuration, Redis integration setup, and scheduling logic for periodic tasks.

The remaining **50–60%** of the project — including architecture design, database schema creation, backend development with Flask, API logic, data models, business logic implementation, and end-to-end integration — was **independently developed**.

All AI-assisted portions were **manually verified, tested, and fully understood** before inclusion in the final implementation.

## DB Schema Design

**Database:** SQLite

**ORM:** SQLAlchemy

### Tables

| Table               | Columns                                                                                                                                                                                                       | Description                                                                |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>User</b>         | <code>id</code> , <code>username</code> , <code>email</code> , <code>password</code> (hash), <code>full_name</code> , <code>role</code> , <code>created_at</code>                                             | Stores all system users (admin, user). Core identity table.                |
| <b>ParkingLot</b>   | <code>id</code> , <code>prime_location_name</code> (unique), <code>address</code> , <code>pin_code</code> , <code>price_per_hour</code> , <code>maximum_spots</code> , <code>created_at</code>                | Contains parking lot information and inventory units managed by admins.    |
| <b>ParkingSpot</b>  | <code>id</code> , <code>lot_id</code> , <code>spot_number</code> (unique per lot), <code>status</code> ('A'), <code>created_at</code>                                                                         | 'O'), <code>created_at</code>                                              |
| <b>Reservation</b>  | <code>id</code> , <code>user_id</code> , <code>spot_id</code> , <code>vehicle_number</code> , <code>parking_timestamp</code> , <code>leaving_timestamp</code> , <code>status</code> , <code>final_cost</code> | Tracks booking lifecycle between users and spots with billing information. |
| <b>ExportJob</b>    | <code>id</code> , <code>job_type</code> , <code>user_id</code> , <code>status</code> , <code>file_path</code> , <code>created_at</code> , <code>finished_at</code>                                            | Records all asynchronous export jobs (CSV) for tracking and download.      |
| <b>Notification</b> | <code>id</code> , <code>user_id</code> , <code>type</code> , <code>title</code> , <code>message</code> , <code>is_sent</code> , <code>scheduled_time</code> , <code>sent_time</code> , <code>file_path</code> | Stores reminders and report notifications sent to users.                   |

### Constraints Applied

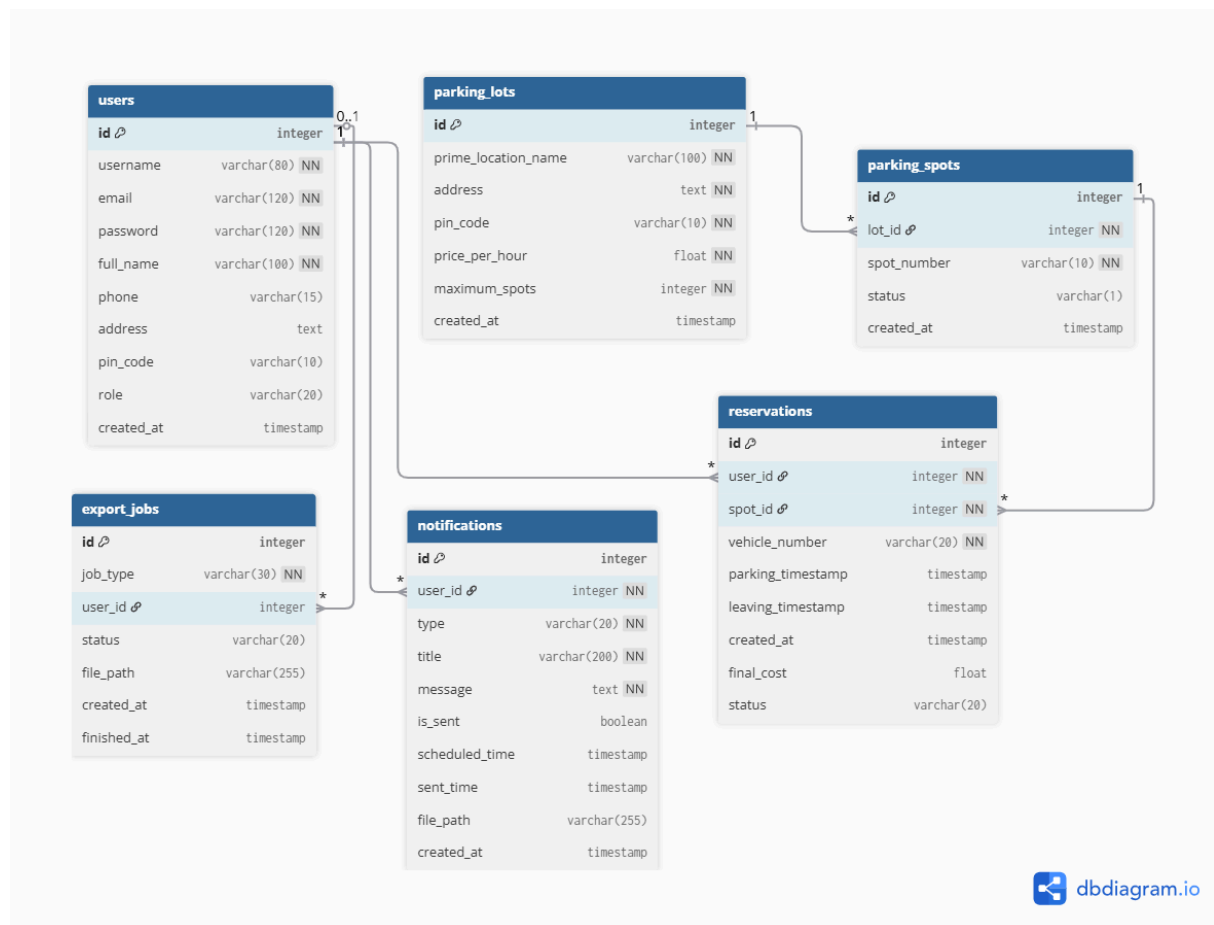
- Foreign key relations maintain referential integrity.

- Unique constraints on usernames/emails and location names.
- **status** restricted to enum values ('A' for Available, 'O' for Occupied).
- **role** restricted to enum values (admin, user).
- Cascade deletions handled programmatically to prevent orphan records.

## Relationships

- **One-to-Many:** **User** → **Reservation** (A user can have multiple reservations)
- **One-to-Many:** **ParkingLot** → **ParkingSpot** (A lot contains multiple spots)
- **One-to-Many:** **ParkingSpot** → **Reservation** (A spot can have multiple reservations over time)
- **One-to-Many:** **User** → **ExportJob** (A user can trigger multiple export jobs)
- **One-to-Many:** **User** → **Notification** (A user can receive multiple notifications)

## ER Diagram



# API Design

Base URL: `/api/`

## Authentication Endpoints

| Endpoint                      | Method | Role   | Functionality                          |
|-------------------------------|--------|--------|----------------------------------------|
| <code>/api/register</code>    | POST   | Public | Register a new user (role=user).       |
| <code>/api/login</code>       | POST   | All    | User login with JWT token generation.  |
| <code>/api/admin/login</code> | POST   | Admin  | Admin login with JWT token generation. |

## User Profile Endpoints

| Endpoint                  | Method | Role       | Functionality                         |
|---------------------------|--------|------------|---------------------------------------|
| <code>/api/profile</code> | GET    | User/Admin | Get current user profile information. |
| <code>/api/profile</code> | PUT    | User/Admin | Update current user profile.          |

## Parking Lot Endpoints

| Endpoint                                      | Method | Role   | Functionality                                    |
|-----------------------------------------------|--------|--------|--------------------------------------------------|
| <code>/api/parking-lots</code>                | GET    | Public | Get all parking lots with availability (cached). |
| <code>/api/admin/lots</code>                  | GET    | Admin  | Get all parking lots (admin view).               |
| <code>/api/admin/lots</code>                  | POST   | Admin  | Create a new parking lot.                        |
| <code>/api/admin/lots/&lt;id&gt;</code>       | PUT    | Admin  | Update parking lot details.                      |
| <code>/api/admin/lots/&lt;id&gt;</code>       | DELETE | Admin  | Delete a parking lot.                            |
| <code>/api/admin/lots/&lt;id&gt;/spots</code> | GET    | Admin  | Get all spots within a specific lot.             |

## Reservation Endpoints

| Endpoint                                          | Method | Role | Functionality                                              |
|---------------------------------------------------|--------|------|------------------------------------------------------------|
| <code>/api/reservations</code>                    | POST   | User | Create a new reservation for an available spot.            |
| <code>/api/reservations/&lt;id&gt;/release</code> | POST   | User | Complete a reservation, calculate cost, and free the spot. |
| <code>/api/user/reservations</code>               | GET    | User | Get user reservation history.                              |
| <code>/api/user/reservations/active</code>        | GET    | User | Get active reservations for the logged-in user.            |

| Endpoint                             | Method | Role  | Functionality                                       |
|--------------------------------------|--------|-------|-----------------------------------------------------|
| <code>/api/admin/reservations</code> | GET    | Admin | View all reservations (booked/completed/cancelled). |

## Analytics Endpoints

| Endpoint                                   | Method | Role  | Functionality                                |
|--------------------------------------------|--------|-------|----------------------------------------------|
| <code>/api/admin/analytics/overview</code> | GET    | Admin | Get admin analytics dashboard data (cached). |
| <code>/api/user/analytics/overview</code>  | GET    | User  | Get user analytics dashboard data (cached).  |

## Export Endpoints

| Endpoint                                                     | Method | Role  | Functionality                                               |
|--------------------------------------------------------------|--------|-------|-------------------------------------------------------------|
| <code>/api/user/export-history</code>                        | POST   | User  | Trigger CSV export of user reservation history (async job). |
| <code>/api/user/export-history/&lt;jobId&gt;</code>          | GET    | User  | Poll export job status.                                     |
| <code>/api/user/export-history/&lt;jobId&gt;/download</code> | GET    | User  | Download completed user export file.                        |
| <code>/api/admin/export-all</code>                           | POST   | Admin | Trigger CSV export of all reservations (async job).         |
| <code>/api/admin/export-all/&lt;jobId&gt;</code>             | GET    | Admin | Poll admin export job status.                               |
| <code>/api/admin/export-all/&lt;jobId&gt;/download</code>    | GET    | Admin | Download completed admin export file.                       |

## Admin Management Endpoints

| Endpoint                      | Method | Role  | Functionality                    |
|-------------------------------|--------|-------|----------------------------------|
| <code>/api/admin/users</code> | GET    | Admin | Get all users list (admin only). |

All APIs are secured with **role-based access control** and implemented using **Flask Blueprints** for separation of concerns.

Each response follows a JSON format with success/failure messages and data payload.

## Architecture and Features

# Architecture Overview

## Backend Structure:

- `backend/ app.py` – Flask application factory, Redis + Celery setup, health/ops endpoints, and CORS configuration.
- `backend/ routes.py` – API resources organized by blueprints (auth, lots/spots, reservations, analytics, exports).
- `backend/ models.py` – SQLAlchemy models with relationships and helper methods.
- `backend/ tasks.py` – Celery tasks for asynchronous operations: CSV exports, daily reminders, and monthly PDF reports.

## Frontend Structure:

- `frontend/src` – Vue 3 SPA with Vite build system.
- Pages and components organized by role (Admin/User dashboards).
- Router guards for authentication and authorization.
- Axios interceptors for JWT token management.

**Database:** SQLite file auto-generated and seeded programmatically with sample data.

**Background Jobs:** Celery integrated with Redis to execute periodic and asynchronous tasks without blocking HTTP requests.

**Caching Strategy:** Redis caching for frequently accessed endpoints (lots list, analytics) with TTL and cache invalidation on updates.

# Features Implemented

## Core Features

### Admin:

- Login to dashboard with role-based authentication.
- Create, update, and delete parking lots.
- View all parking spots within each lot with status indicators.
- View all users in the system.
- View all reservations (active, completed, cancelled) across all users.
- Dashboard statistics: total lots, total spots, total users, total reservations, revenue metrics.
- Analytics dashboard with charts showing utilization and revenue trends.
- Export all reservation data as CSV (async job with status tracking).

## User:

- Self-register and login with JWT authentication.
- View all available parking lots with real-time availability.
- Browse parking spots by lot and check availability.
- Create reservations by selecting available spots and entering vehicle details.
- Release/complete active reservations with automatic cost calculation.
- View reservation history with filtering options.
- View active reservations separately.
- Personal analytics dashboard showing usage patterns and spending.
- Export personal reservation history as CSV (Celery async job).

## Background Jobs

- **Daily Reminders:** Automated daily task that sends appointment notifications to users about active reservations via Google Chat webhook or email fallback.
- **Monthly Reports:** Scheduled task that generates and emails monthly PDF reports with reservation summary per user.
- **CSV Export:** User-triggered asynchronous job for exporting reservation history, preventing UI blocking during large data exports.

## User Experience Features

- **Responsive UI:** Bootstrap 5 utilities ensure mobile-friendly interface.
- **Loading States:** Spinners and status indicators during async operations.
- **Auto-download:** Background polling for export jobs with automatic download when ready.
- **Real-time Updates:** Cached data with automatic invalidation ensures fresh information.
- **Form Validation:** Client-side and server-side validation for all inputs.
- **Error Handling:** User-friendly error messages and fallback states.

## Caching

- Redis caching for repeated queries:
  - Public parking lots list with availability (30-minute TTL).
  - Analytics overview data (15-minute TTL).
  - Cache invalidation on lot/spot/reservation updates.



- Significant performance improvement for frequently accessed endpoints.
- 

## Video Presentation

**Drive Link:** [Please click here](#)